Guia Completo de Desenvolvimento de Software — Squad MOD-2

Complemento: Planejamento da Sprint 3 — Integração e Armazenamento em Nuvem

> Complemento do guia docs/responsability.md / `docs/GUIA_DESENVOLVIMENTO_MOD2.pdf` (seções 1–9). Estende as **seções 4 (Diretrizes de Arquitetura e Submódulos)**, **5 (Arquitetura de Código, DDD)**, **6 (Padrões GoF)** e **9 (Checklist do Desenvolvedor)** para o ciclo de integração entre componentes e substituição do disco físico por armazenamento em nuvem. Referências externas: `docs/ODS 2026_2 – Arquitetura 6c.html` (Especificação de interfaces 1.0) e `docs/Guia_Kafka_B2 (1).PDF`.

---

10. Planejamento da Sprint 3

10.1 Objetivo e escopo

A Sprint 3 fecha a lacuna entre os componentes já implementados isoladamente (S4 em `ods-26-2/s4`, S5 em `ods-26-2/s5`, B6 em `ods-26-2/b6`) e a arquitetura 6c, em dois eixos:

1. **Integração dos componentes** entre si e com o **barramento B2/B3** (Kafka), conforme o Guia de Uso do Kafka.
2. **Armazenamento em nuvem (Google Cloud)** substituindo o disco físico para as mídias probatórias do S4, mantendo os metadados localmente.

Aplica-se a todo o Squad MOD-2 (Davi Gomes, Henrique Azevedo, Rafael Castro, Eduardo Gomes), na divisão de responsabilidades da seção 3 do guia.

10.2 Análise de aderência à arquitetura 6c

O que a 6c muda para o MOD-2:

- **S4 passa a consumir S2** e grava automaticamente todo `ods.s2.event` com `status` `CONFIRMED` / `UNCERTAIN` e `phase` `STARTED`, com a pré-rolagem configurada.
- S4 atende **`ods.s4.capture.command`** (reply imediato `{"status":"OK","clip_id":…}`; descritor completo por `ods.s4.clip`) e **`ods.s4.clip.query/.reply`** (por `clip_id` ou `event_id`; lista vazia se expirou).
- O descritor **`ods.s4.clip`** tem `media_type` (CLIP/SNAPSHOT) e **`expires_at`** **obrigatório**; `media_uri` é referência (ex.: `s4://clips/AAAA/MM/DD/<uuid>.mp4`), nunca o binário.
- S4 consome também **`ods.session.started`** (troca de sessão zera estado e `sequence`) e os streams `ods.p4.frame` / `ods.p5.audio_chunk` (metadados fora do barramento).
- **S5** consome `ods.s1.*`, `ods.s2.*` e I3; produz `ods.s5.zone_metrics` e `ods.s5.event_metrics`; atende `ods.s5.metrics.query/.reply` e `ods.s5.heatmap.query/.reply`.

- **B6** é camada 4 embutida nas aplicações; o editor de zonas alimenta o artefato **ZONES** (formato §8 da spec).

O que o Guia Kafka (B2) exige:

- Broker em Google Cloud `34.66.235.91:9093` é **temporário** — o endereço tem de ser **configurável** (flag/env), nunca fixo no código.
- Cliente por linguagem: C++ → **librdkafka**; Python → **confluent-kafka**; Node/TS → kafkajs.
- Validação do envelope B1 em três níveis (JSON válido → campos obrigatórios/schema/versão → payload); fora do formato: **descartar e registrar**.
- Idempotência por `message_id`; nenhum binário no payload (só referências); `sequence` para detecção de perda.
- **Créditos GCP**: sem laços sem condição de parada, poll ≥ 1 s, encerrar processos de teste, combinar carga com a squad TRA-1.

Estado atual (ponto de partida):

Componente	Estado
`ods-26-2/s4` (C++)	Completo: ring buffer real (GStreamer), binding evento-mídia, retenção/expurgo LGPD, REST (cpp-httplib), Docker multi-stage. `B3EventConsumer` é **só contrato**.
`ods-26-2/s5` (Python)	Clean Architecture com DuckDB/FastAPI; consumidor `InMemoryB3EventConsumer` e envelope antigo (`event_type/origin/coords/timestamp`); sem Kafka real.
`ods-26-2/b6` (TypeScript)	OverlayPlayer e ZoneEditor com testes Vitest; **mockados**, sem ligação com S4 nem export de artefato ZONES.

10.3 Decisões da sprint

Tema	Decisão
Escopo de integração	Todos os componentes: **S4 + S5 + B6 + barramento Kafka**
Nuvem	**Clipes/snapshots em GCS**; metadados continuam no **SQLite local**
Cliente GCS (C++)	**REST JSON API via cpp-httplib (vendado) + JWT service account (RS256/OpenSSL)**
Kafka	**Integração real ponta a ponta** (librdkafka), bootstrap configurável
URI do descritor	Mantém `s4://clips/...` canônica no DTO; `GcsFileStorage` mapeia para a chave do bucket; o consumidor recebe **URL assinada V4** de curta duração
Storage local	`--storage local` continua disponível (testes/offline); híbrido: grava local → upload assíncrono → publica `ods.s4.clip` só após sucesso

10.4 Trabalhos (WIs) e responsáveis

Em linha com a seção 3 do guia (bounded context + especialização de stack).

Frente A — S4 + Kafka (Davi, integração)

WI	Entregável
A1	**Envelope B1 em C++** (`infrastructure/b3_bus/envelope.{hpp,cpp}`): builder dos 13 campos (perfis EV-SERVIÇO, CMD, RPL), validador em 3 níveis, `sequence` por producer+session, dedup por `message_id` .
A2	`KafkaEventTransport` (implementa o contrato `B3EventConsumer` da seção 6 do guia — Observer/Consumer): librdkafka, bootstrap de `--kafka-brokers` /env, `group.id` próprio (`S4-…`), `auto.offset.reset=earliest` , poll ≥ 1 s, `stop()` no SIGTERM. Consome `ods.s2.event` , `ods.session.started` , `ods.s4.capture.command` , `ods.s4.clip.query` .
A3	**Consumo → casos de uso**: `ods.s2.event` (CONFIRMED/UNCERTAIN + STARTED) dispara `ExtractClipUseCase` com pré-rolagem (`ConsumeS2EventUseCase`); `capture.command` → reply imediato + descritor por `ods.s4.clip` ; `clip.query/.reply` sobre o repositório.
A4	**Produtor**: publicar `ods.s4.clip` (EV-SERVIÇO, `correlation_id` = event_id/comando) ao concluir a extração; replies RPL para command/query. **Risco**: naming de tópico de reply não consta no guia → alinhar com TRA-1 (padrão provisório `ods.s4.<nome>.reply`).
A5	`main.cpp` : flags `--kafka-brokers` , `--kafka-group-id` , `--storage local|gcs` , `--gcs-bucket` , `--gcs-credentials` ; transporte desabilitado se ausente (POST `/api/v1/events` continua funcionando).

Frente B — Nuvem (Davi e Henrique, substitui o disco físico)

WI	Entregável
B1	`GcsFileStorage : IFileStorage` (`infrastructure/gcs/`): `write/exists/sizeOf/remove` via JSON API (`uploadType=media` , `GET/DELETE /storage/v1/b/…/o/…`); OAuth2 com JWT RS256 (OpenSSL) → `oauth2.googleapis.com/token` , com **cache e refresh do token**; bucket/key derivados de `media_uri` .
B2	**URL assinada V4** (`GcsUrlSigner`): assinatura RSA do canonical request → HTTPS temporário (TTL ~15 min) para B6/app reproduzir o clipe.
B3	**Upload resumable** (`uploadType=resumable`) para arquivos ≥ 5 MB — evita estouro de limite e falha parcial.
B4	**Retenção/expurgo na nuvem — responsável: Henrique** (evolução do seu S4.3): `StatvfsDiskUsageProvider` não vale para bucket → `IBucketUsageProvider` (JSON API `GET /storage/v1/b/<bucket>` → `storage.bytesUsed`) para a cota FIFO; `PurgeMediaUseCase` remove do GCS via `IFileStorage` ; **lifecycle rule** no bucket como rede de segurança (apagar após `expires_at`); `expires_at` = retenção (7 dias LGPD).
B5	**Migração/fallback**: `--storage local` mantido; híbrido grava local → upload assíncrono → só publica `ods.s4.clip` após sucesso (retry limitado).

Frente C — S5 integrado (Rafael, `ods-26-2/s5`)

WI	Entregável
C1	**Adaptar ao envelope B1/6c**: parsing dos 13 campos; assinar `ods.s1.zone_transition` , `ods.s1.zone_occupancy` , `ods.s2.event` (hoje espera `event_type/origin/coords/timestamp`). Aceitar os dois formatos temporariamente para não quebrar os 59 testes.
C2	`KafkaB3EventConsumer` **(confluent-kafka)** substituindo o `InMemory` (mantido nos testes): grupo `S5-…` , bootstrap configurável, validação/descarte com log, idempotência.
C3	**Producer/reply**: publicar `ods.s5.zone_metrics` e `ods.s5.event_metrics` ; atender `ods.s5.metrics.query/.reply` e `ods.s5.heatmap.query/.reply` .

WI	Entregável
C4	**Artefatos**: carregar ZONES/CALIBRATION na partida (formato §8 da spec) em vez de arquivo solto de homografia.

Frente D — B6 integrado (Eduardo, `ods-26-2/b6`)

WI	Entregável
D1	**OverlayPlayer ↔ S4**: cliente `S4ClipClient` (REST: `GET /api/v1/clips/{id}`, `POST /api/v1/events`) obtendo descritor + URL assinada, reproduzindo e sincronizando `points_of_interest` na timeline.
D2	**ZoneEditor → artefato ZONES**: export no formato §8 (`space_id`, `coordinates`, `source_ids`, `zones[].polygon/enter_hysteresis_ms/exit_hysteresis_ms/max_occupancy/classes`), pronto para S1/S5.
D3	**Demos reais**: páginas `index.html` apontando para o S4 rodando (Vite + proxy), sem mocks.

10.5 Cronograma (2 semanas)

Fase	Dias	Entregável
1	1–2	WI-A1 envelope + testes unit · WI-B1 `GcsFileStorage` + fakes · WI-C1
2	3–5	WI-A2/A3 transporte + consumo · WI-B2/B4 URL assinada + expurgo GCS · WI-C2
3	6–8	WI-A4/A5 produtor + replies · WI-B5 híbrido/upload · WI-C3 · WI-D1
4	9–10	WI-D2/D3 · **E2E do squad** · docs + Docker

10.6 Verificação (Definition of Done)

Complementa a seção 8 (Estratégia de Testes) e a seção 9 (Checklist) do guia.

```
# S4 (C++)
cmake -S mod2/s4-media-evidence -B build && cmake --build build
ctest --test-dir build --output-on-failure

# S5 (Python)
pytest

# B6 (TypeScript)
npx vitest
```

Smoke E2E do squad (script novo `scripts/sprint3_smoke.sh`):

1. Publicar `ods.s2.event` (CONFIRMED/STARTED) no broker do guia → S4 extrai o clipe, sobe no bucket e publica `ods.s4.clip`.
2. `GET /api/v1/clips/{id}` → `media_uri` + `expires_at` + hash; URL assinada baixa o MP4 com SHA-256 idêntico.
3. `ods.s4.capture.command` (SNAPSHOT) → reply OK + descritor; `ods.s4.clip.query` por `event_id` → lista.
4. S5 consome os mesmos eventos → `GET /api/v5/metrics/{zone_id}` com dados reais.
5. B6 OverlayPlayer reproduz o clipe pela URL assinada e exibe POIs; ZoneEditor exporta ZONES válido.

6. Expurgo: `expires_at` vencido → objeto removido do bucket + `is_retained=false` no SQLite.

10.7 Riscos e cuidados

- **Créditos GCP**: testes contra o broker real curtos e supervisionados, poll ≥ 1 s, encerrar processos (guia §4.5); scripts de carga combinados com TRA-1.
- **Reply de command/query**: naming de tópico não definido no guia → alinhar com TRA-1 (dono de B1/B3).
- **Credenciais**: service account fora do repositório (`.gitignore`), sem logar tokens; rotação combinada com a equipe.
- **librdkafka no Docker/Jetson**: `librdkafka-dev` no builder e `librdkafka1` no runtime (amd64+arm64 no apt); flag `ODS_S4_WITH_KAFKA=ON/OFF` , como a do GStreamer.
- **LGPD na nuvem**: regra dos 7 dias não muda — `expires_at` + expurgo removendo do bucket; bucket autenticado (sem acesso público) e URL assinada curta.
- **Fuso/hora**: tudo ISO 8601 UTC com milissegundos; `sequence` / `session_id` zerados em `ods.session.started` .
- **Quebra de contrato do S5**: aceitar envelope antigo e 6c temporariamente (dual-mode) para não travar a suíte existente.

10.8 Rastreabilidade com o guia

Seção do guia	Como este complemento estende
§4.1 (S4)	Adiciona consumo automático de `ods.s2.event` , `capture.command` , `clip.query` e a fonte de mídia em GCS no lugar do NVMe.
§4.2 (S5)	Troca o consumidor in-memory pelo Kafka real e liga a produção às métricas da 6c.
§4.3 (B6)	Liga OverlayPlayer ao S4 (descritor + URL assinada) e formaliza o artefato ZONES na saída do ZoneEditor.
§5 (DDD/Clean Architecture)	Novos adaptadores em `infrastructure/` (`gcs/` , `b3_bus/`) atrás dos contratos de domínio `IFileStorage` / `B3EventConsumer` ; domínio intocado.
§6 (Padrões GoF)	`GcsFileStorage` e `FileStorage` como Strategy de armazenamento; `KafkaEventTransport` mantém o papel Observer/Consumer.
§8 (Testes)	Fakes em memória para GCS e Kafka garantem testes unitários sem I/O; E2E do squad cobre a integração real.
§9 (Checklist)	Acrescentam-se: bootstrap configurável (sem endereço fixo), sem binário no barramento, idempotência por `message_id` e conformidade LGPD com `expires_at` refletido no bucket.